ASTRA

A Uniform Timing Processor

Architecture Design & Functional Verification

Implemented in SystemVerilog
Simulated in Xilinx Vivado 2019.1

Dhruv Choudhari

EECE [AI/ML]

Table of Contents

Table of Contents 2
1. Introduction 5
2. Architecture Overview 6
2.1 Key Parameters 6
2.2 Instruction Format 7
2.3 Datapath Summary 7
3. Module Descriptions 8
3.1 Clock Generator (Clock) 8
Inputs 8
Outputs 8
3.2 Counter (Program Counter) 8
Inputs 8
Outputs 8
3.3 Programme Block 9
Inputs 9
Outputs 9
3.4 Register File 9
Inputs 9
Outputs 9
3.5 ALU (Arithmetic Logic Unit) 10
Inputs 10
Outputs 10
Supported Operations 10
3.6 Conditional Unit (Cond) 11
Inputs 11
Outputs 11
Supported Conditions 11
3.7 Xtra Unit (Shift/Rotate) 12
Inputs 12
Outputs 12
Supported Operations 12
3.8 Stack 12
Inputs 12
Outputs 13
3.9 RAM 13

Inputs 13
Outputs 13
4. Instruction Set Reference 14
4.1 Calculation Mode (op_code[7] = 0) 14
ALU Instructions (bits[5:4] = 00) 14
Conditional / Branch Instructions (bits[5:4] = 01) 14
Shift/Rotate Instructions (bits[5:4] = 10) 15
Stack Instructions (bits[5:4] = 11) 15
4.2 Move Mode (op_code[7] = 1) 15
5. Design Decisions & Tradeoffs 16
6. Simulation & Verification 16
6.1 Clock Generator 18
Signals monitored 18
Waveform 18
Result 18
6.2 Counter (Program Counter) 18
Signals monitored 18
Waveform 19
Result 19
6.3 Register File 19
Signals monitored 19
Waveform 19
Observations 19
Result 20
6.4 ALU 20
Signals monitored 20
Waveform 20
Result 20
6.5 Conditional Unit 20
Signals monitored 21
Waveform 21
Result 21
6.6 Stack 21
Signals monitored 21
Waveform 21
Result 22

6.7 Full System - Top-Level Integration 22
Signals monitored 22
Waveform 22
Result 22
7. Known Limitations 23
7.1 Program Memory Capacity 23
7.2 Memory Addressing 23
7.3 Move Mode Encoding Efficiency 23
7.4 No Hardware Call/Return 23
7.5 Simulation-Only Clock 23
7.6 No Reset Signal 23
8. Conclusion 24
Appendix A: Instruction Encoding Quick Reference 25
op_code Bit Layout 25
Register Index Encoding 25
Flag Register Layout 25
9. Module Wise Code 26

1. Introduction

Astra is a minimal experimental processor architecture designed to explore fundamental principles of processor design, including datapath organization, instruction semantics, and control logic. The architecture prioritizes simplicity and deterministic execution in order to make the relationship between hardware structure and instruction behavior easier to analyze and understand.

Astra follows a single-cycle execution model, in which every instruction completes within a single clock cycle. This uniform timing model simplifies the control logic and avoids the need for pipeline hazard detection or stall logic, making the architecture well-suited as a study platform for understanding processor fundamentals.

The processor supports a compact set of operations including:

- Basic arithmetic and logical operations (unsigned and signed)
- Conditional control flow via flag-based branching
- Simple memory access (load and store)
- Stack operations (push, pop to register, pop to program counter)
- Shift and rotate operations

The architecture has been implemented and tested using SystemVerilog, with functional verification performed through simulation in Xilinx Vivado. Simulation results demonstrate correct execution of basic programs within the constraints of the architecture.

2. Architecture Overview

Astra is an 8-bit single-cycle processor with a flat memory model and a fixed 3-byte instruction format. Every instruction is encoded as three consecutive bytes in program memory: an op-code byte, a first operand byte (Op1), and a second operand byte (Op2). The program counter advances by three after each instruction, except when a branch or jump redirects it.

2.1 Key Parameters

Signal	Description
`Data width`	8 bits
`Register count`	8 general-purpose registers (r0-r7)
`Program memory`	256 bytes (up to ~85 instructions)
`Data memory (RAM)`	256 bytes, byte-addressable
`Stack depth`	256 entries, 8-bit wide
`Clock period`	2 ns (500 MHz, simulation only)
`Instruction size`	3 bytes (op_code, op1, op2)
`PC increment`	+3 per cycle (non-branch)
`Flag register`	4 bits: zero [3], overflow [2], negative [1], carry [0]

2.2 Instruction Format

All instructions are encoded in three bytes:

Byte 0: op_code	Byte 1: op1	Byte 2: op2
Encodes operation, mode, and unit select	*Source register / immediate value / jump address*	*Destination register / second source register*

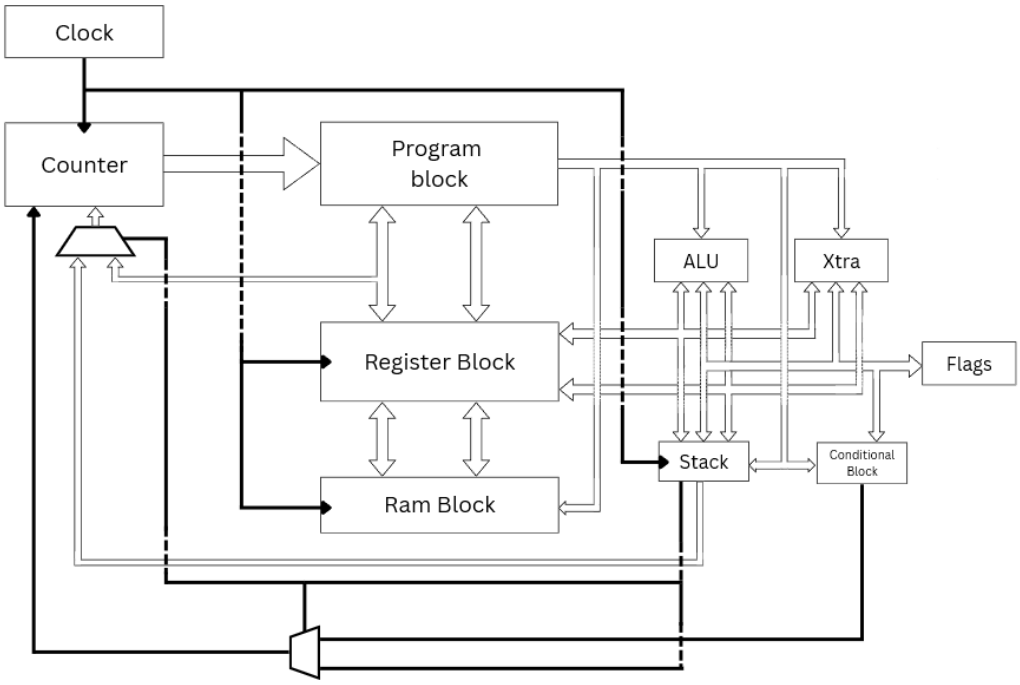
The op_code byte is further subdivided as follows:

- Bit [7]: mode select - 0 = calculation mode, 1 = move (MOV) mode
- Bits [5:4]: unit select (calculation mode only) - 00 = ALU, 01 = Conditional, 10 = Xtra, 11 = Stack
- Bits [3:0]: operation code within the selected unit
- Bit [6]: unused in the current revision (reserved for future use)

2.3 Datapath Summary

The Top module acts as the central decode and routing hub. On every clock cycle, the Programme block reads three bytes from program memory at the current PC address and presents them as op_code, op1, and op2. The Top module decodes op_code combinationally and asserts enable signals to exactly one functional unit per cycle. Results are written back to the register file or fed to the program counter on the next clock edge.

(DataFlow Diagram)



3. Module Descriptions

3.1 Clock Generator (Clock)

The Clock Generator provides the timing signal used to synchronize all synchronous components within the processor. It produces a periodic clock signal with a 2 nanosecond period, corresponding to a simulated frequency of 500 MHz. This module is simulation-only and would be replaced by an external clock input or PLL in a synthesizable design.

Inputs

None.

Outputs

Signal	Description
`clk`	Periodic clock signal, period = 2 ns (500 MHz simulated)

3.2 Counter (Program Counter)

The Counter module implements the Program Counter (PC). It increments by 3 on every clock cycle to advance through the 3-byte instruction stream, and loads a new address when a jump or branch is taken. The initial value of the counter is 0, pointing to the first instruction in program memory.

Inputs

Signal	Description
`jmp_en`	Jump enable. When asserted, loads jmp into the counter instead of incrementing.
`jmp[7:0]`	Target address to load when jmp_en is asserted.
`clk`	Clock input. Counter updates on the rising edge.

Outputs

Signal	Description
`PC[7:0]`	Current program counter value, updated each rising clock edge.

3.3 Programme Block

The Programme block contains a 256-byte memory array that holds the executable program. It presents three consecutive bytes from the current PC address as combinational outputs on every clock cycle: the operation code, the first operand, and the second operand. The program is loaded into memory at simulation start from an external binary file (programme.mem).

Inputs

Signal	Description
`count[7:0]`	Pointer to the address of the current instruction (from Counter/PC).

Outputs

Signal	Description
`op_code[7:0]`	Operation code for the current instruction (prog[count]).
`op1[7:0]`	First operand of the current instruction (prog[count+1]).
`op2[7:0]`	Second operand of the current instruction (prog[count+2]).

3.4 Register File

The Register File provides 8 general-purpose 8-bit registers (r0 through r7). Reads are asynchronous - output values update immediately when the select lines change. Writes are synchronous and occur on the rising edge of the clock when the write enable signal is asserted.

Inputs

Signal	Description
`w_en`	Write enable. When asserted, saves the value on Save into regs[Sav_Loc] on the next rising clock edge.
`save[7:0]`	Data value to write into the register file.
`sav_loc[2:0]`	Destination register index for write operations (0-7).
`sel1[2:0]`	Select line for read port 1. Determines the register value sent to Out1.
`sel2[2:0]`	Select line for read port 2. Determines the register value sent to Out2.
`clk`	Clock input. Write operations are clocked on the rising edge.

Outputs

Signal	Description
`out1[7:0]`	Value of the register selected by sel1. Asynchronous read.

`out2[7:0]`	Value of the register selected by sel2. Asynchronous read.

3.5 ALU (Arithmetic Logic Unit)

The ALU performs arithmetic and logical computations. It handles both unsigned and signed operations, with the operation mode encoded in the op_code. The ALU updates a 4-bit flags output on every enabled operation. The flag layout is: bit[3] = zero (Z), bit[2] = overflow (V), bit[1] = negative (N), bit[0] = carry (C).

Inputs

Signal	Description
`en`	Enable signal. ALU outputs are active only when asserted.
`Op_code[3:0]`	Selects the arithmetic or logical operation to perform.
`op1[7:0]`	First operand (typically the destination register value).
`op2[7:0]`	Second operand.
`flagsin[3:0]`	Current flags register value (used by ADC and SBB operations).

Outputs

Signal	Description
`out[7:0]`	Result of the operation.
`flagsout[3:0]`	Updated flags: [3]=Z, [2]=V, [1]=N, [0]=C.

Supported Operations

Signal	Description
`op_add (0000)`	Unsigned addition: op1 + op2. Sets carry and zero flags.
`op_sub (0001)`	Unsigned subtraction: op1 - op2. Sets carry (borrow) and zero flags.
`op_and (0010)`	Bitwise AND. Sets negative and zero flags.
`op_or (0011)`	Bitwise OR. Sets negative and zero flags.
`op_not (0100)`	Bitwise NOT of op1. Sets negative and zero flags.
`op_xor (0101)`	Bitwise XOR. Sets negative and zero flags.
`op_cmp (0110)`	Compare (op1 - op2), updates flags only, no writeback.
`op_adds (1000)`	Signed addition. Sets carry, overflow, negative, and zero flags.
`op subs (1001)`	Signed subtraction. Sets carry, overflow, negative, and zero flags.

op_adc (1010)	Add with carry: op1 + op2 + C. Unsigned multi-word arithmetic.
op sbb (1011)	Subtract with borrow: op1 - op2 - ~C. Unsigned multi-word arithmetic.

3.6 Conditional Unit (Cond)

The Conditional unit evaluates branch conditions based on the current state of the flags register and asserts a jump signal when the specified condition is met. All branch target addresses are supplied as immediate values in the op1 field of the instruction. The conditional unit clears the flags register after evaluation, as flags are not required to persist across a jump in this architecture.

Inputs

Signal	Description
en	Enable signal.
Op_code[3:0]	Branch condition selector.
flagsin[3:0]	Current flags: [3]=Z, [2]=V, [1]=N, [0]=C.

Outputs

Signal	Description
jmp	Asserted when the branch condition is satisfied.
flagsout[3:0]	Always outputs 4'b0000 (flags cleared after branch evaluation).

Supported Conditions

Signal	Description
op_jmp (0000)	Unconditional jump.
op_jnz (0001)	Jump if not zero: Z = 0.
op_jez (0010)	Jump if zero: Z = 1.
op_less (0011)	Jump if less than: N XOR V = 1 (signed comparison).
op_lesse (0100)	Jump if less than or equal: (N XOR V) OR Z.
op_great (0101)	Jump if greater than: Z = 0 AND (N XOR V) = 0.
op greate(0110)	Jump if greater than or equal: Z = 1 OR (N XOR V) = 0.

3.7 Xtra Unit (Shift/Rotate)

The Xtra unit handles shift and rotate operations. The shift/rotate amount is encoded in the lower 3 bits of op2, allowing shifts from 0 to 7 positions. Results are written back to the register specified in op1.

Inputs

Signal	Description
`en`	Enable signal.
`op[2:0]`	Operation selector (xtr_op enum).
`op1[7:0]`	Value to be shifted or rotated.
`op2[2:0]`	Shift/rotate amount (lower 3 bits used).
`flagsin[3:0]`	Current flags (passed through to flagsout).

Outputs

Signal	Description
`out[7:0]`	Result of the shift or rotate.
`flagsout[3:0]`	Flags passed through from flagsin (no flag modification).

Supported Operations

Signal	Description
`op_shr (000)`	Logical shift right by op2[2:0] positions.
`op_shl (001)`	Logical shift left by op2[2:0] positions.
`op_ror (010)`	Rotate right by op2[2:0] positions (wraps bits around).
`op_rol (011)`	Rotate left by op2[2:0] positions (wraps bits around).

3.8 Stack

The Stack module provides a 256-entry, 8-bit wide last-in-first-out (LIFO) storage structure. It is controlled by a push/pop select signal (pp) and an enable signal. On pop, the output can be routed either to the register file (for general value retrieval) or directly to the program counter (for return-from-subroutine behavior), as determined by op_code[1] in the Top module.

Inputs

Signal	Description
`en`	Enable signal. Stack operations occur only when asserted.
`pp`	Push/pop select: 0 = push, 1 = pop.
`val[7:0]`	Value to push onto the stack (from register file output).
`clk`	Clock input. All stack operations are synchronous.

Outputs

Signal	Description
`out[7:0]`	Top-of-stack value on pop. Routed to register file or PC by Top module.

3.9 RAM

The RAM module provides 256 bytes of general-purpose read/write data memory. Write operations are synchronous (clocked). Read operations are combinational - the output is valid as long as the enable and read signals are asserted and the address is stable. The address is supplied as the value held in the register pointed to by op1, making all memory accesses register-indirect.

Inputs

Signal	Description
`en`	Enable signal.
`read`	Read/write select: 1 = read, 0 = write.
`s[7:0]`	Memory address (register-indirect, supplied from register file output).
`sv[7:0]`	Data value to write (store operations).
`clk`	Clock input. Write operations are clocked.

Outputs

Signal	Description
`out[7:0]`	Data read from the memory address specified by s. Valid when en=1 and read=1.

4. Instruction Set Reference

All instructions occupy 3 bytes in program memory. The op_code byte fully specifies the operation; op1 and op2 carry operands whose roles depend on the instruction type. Register indices are 3-bit values (0-7) encoded in the lower bits of the operand bytes.

4.1 Calculation Mode (op_code[7] = 0)

Bits [5:4] of op_code select the functional unit. The operation within the unit is encoded in bits [3:0].

ALU Instructions (bits[5:4] = 00)

Opcode (hex)	Mnemonic	Mode	Description
0x00	ADD	Unsigned	op1_reg = op1_reg + op2_reg. Sets C, Z.
0x01	SUB	Unsigned	op1_reg = op1_reg - op2_reg. Sets C (borrow), Z.
0x02	AND	Logical	op1_reg = op1_reg & op2_reg. Sets N, Z.
0x03	OR	Logical	op1_reg = op1_reg | op2_reg. Sets N, Z.
0x04	NOT	Logical	op1_reg = ~op1_reg. Sets N, Z.
0x05	XOR	Logical	op1_reg = op1_reg ^ op2_reg. Sets N, Z.
0x06	CMP	Unsigned	op1_reg - op2_reg, flags updated only. No writeback.
0x08	ADDS	Signed	op1_reg = op1_reg + op2_reg. Sets C, V, N, Z.
0x09	SUBS	Signed	op1_reg = op1_reg - op2_reg. Sets C, V, N, Z.
0x0A	ADC	Unsigned	op1_reg = op1_reg + op2_reg + C.
0x0B	SBB	Unsigned	op1_reg = op1_reg - op2_reg - ~C.

Conditional / Branch Instructions (bits[5:4] = 01)

Opcode (hex)	Mnemonic	Mode	Description
0x10	JMP	Branch	Unconditional jump to immediate address in op1.
0x11	JNZ	Branch	Jump to op1 if Z = 0.
0x12	JEZ	Branch	Jump to op1 if Z = 1.
0x13	JLT	Branch	Jump to op1 if N XOR V = 1 (signed less than).
0x14	JLTE	Branch	Jump to op1 if (N XOR V) OR Z (signed less or equal).

0x15	JGT	Branch	Jump to op1 if Z=0 AND N XOR V=0 (signed greater than).
0x16	JGTE	Branch	Jump to op1 if Z=1 OR N XOR V=0 (signed greater or equal).

Shift/Rotate Instructions (bits[5:4] = 10)

Opcode (hex)	Mnemonic	Mode	Description
0x20	SHR	Xtra	op1_reg = op1_reg >> op2[2:0]. Logical right shift.
0x21	SHL	Xtra	op1_reg = op1_reg << op2[2:0]. Logical left shift.
0x22	ROR	Xtra	op1_reg rotated right by op2[2:0] positions.
0x23	ROL	Xtra	op1_reg rotated left by op2[2:0] positions.

Stack Instructions (bits[5:4] = 11)

Opcode (hex)	Mnemonic	Mode	Description
0x34	PUSH	Stack	Push op1_reg value onto stack. sp++.
0x37	POP	Stack	Pop top of stack into op2_reg. sp--. (op_code[1]=1, op_code[0]=1)
0x35	POPC	Stack	Pop top of stack into Program Counter (return). (op_code[1]=0, op_code[0]=1)

4.2 Move Mode (op_code[7] = 1)

Move mode is selected when bit[7] of op_code is set. Bits [1:0] determine the transfer direction. All memory accesses are register-indirect (the register value is used as the address).

Opcode (hex)	Mnemonic	Mode	Description
0x80	MOV r,r	Move	op2_reg = op1_reg. Register to register transfer.
0x81	STR r,m	Move	RAM[op1_reg] = op2_reg. Store register to memory address.
0x82	LDR m,r	Move	op2_reg = RAM[op1_reg]. Load memory address into register.
0x83	MOV imm,r	Move	op2_reg = op1 (immediate literal). Load constant into register.

5. Design Decisions & Tradeoffs

Single-Cycle Architecture

- Astra implements a **single-cycle architecture**, where every instruction is executed within one clock cycle.

This design choice was made to **prioritize simplicity over performance**. A single-cycle model removes the need for complex control logic, instruction scheduling, and pipeline hazard handling, making the processor easier to design, debug, and reason about.

However, this comes with trade-offs. The processor clock frequency is limited by the **slowest instruction**, as all operations must complete within the same cycle. Additionally, the architecture does not support **pipelining** or **multi-cycle optimizations**, which limits overall performance.

Fixed Instruction Length

- Astra uses a **fixed instruction length of 3 bytes per instruction**.

Every instruction, regardless of its operand requirements, is encoded into this fixed size. Instructions that require fewer operands leave unused fields reserved, while those requiring immediates utilize the available operand fields.

This decision was made to maintain **consistent instruction decoding** and preserve the **single-cycle execution model**. Earlier iterations of Astra used a **4-byte instruction format**, but after refining the instruction set and operand requirements, this was reduced to **3 bytes**. This change improved **code density** while maintaining decoding simplicity and architectural consistency.

Hardwired Control Logic

- Astra uses **hardwired control logic implemented using combinational decoding** for instruction execution.

This decision was made to **prioritize simplicity, lower hardware overhead, and faster instruction decoding**. In this approach, instruction decoding is performed using combinational logic derived directly from the opcode and operand fields, eliminating the need for a microcoded control unit.

This design aligns naturally with Astra's **fixed instruction length** and **single-cycle execution model**, as both benefit from deterministic control signals and simplified decoding paths. Since control signals are generated through predefined combinational logic, instruction execution behaves as a direct mapping between opcode and hardware control paths.

However, this approach introduces certain limitations. Hardwired control logic reduces flexibility and makes **future architectural extensions more difficult**, as modifying instruction behavior requires redesigning combinational logic rather than updating microcode.

Additionally, as the architecture scales, **control complexity increases**, which can make timing closure more difficult and potentially introduce **race conditions or critical path delays**. While these issues are manageable at Astra's current scale, they may limit scalability in more complex future iterations.

Despite these trade-offs, hardwired control was chosen to maintain Astra's goals of **simplicity, determinism, and minimal hardware complexity**.

Lack of a Multiplication and Division Commands

- Astra does not include dedicated **multiplication (MUL)** or **division (DIV)** instructions within its ALU.

This is an intentional design decision driven by Astra's **single-cycle execution model**. Multiplication and division operations are inherently more complex than basic arithmetic operations and are typically implemented using **multi-cycle logic**, iterative algorithms, or dedicated hardware units.

In a strict single-cycle architecture, implementing multiplication or division would require either:

- Large combinational multipliers/dividers, which significantly increase hardware complexity and critical path delay, or
- Multi-cycle iterative logic, which would break Astra's uniform single-cycle execution model.

To maintain **timing uniformity and architectural simplicity**, these instructions were excluded from the initial design.

Despite the lack of dedicated instructions, multiplication and division can still be implemented in software using **repeated addition, subtraction, and shift-based algorithms**, albeit at the cost of increased instruction count and execution time. This preserves functional completeness while keeping hardware complexity low.

The absence of MUL/DIV reduces ALU complexity and helps maintain a predictable clock cycle, but limits computational efficiency for arithmetic-intensive operations. Future iterations of Astra may introduce **multi-cycle execution support** or dedicated arithmetic units to incorporate these operations without compromising architectural coherence.

6. Simulation & Verification

Each module was verified individually through unit-level simulation in Xilinx Vivado XSim before integration. The full system was subsequently verified using a top-level testbench that executes a representative test program covering all major execution paths. Simulation was performed at the behavioral/RTL level with a 1 ns time resolution.

The test program used for system-level verification covers: immediate loads, unsigned addition, compare and conditional branch, memory store and load, and stack push/pop-to-register operations. Waveform results for each simulation are documented in the sections below.

6.1 Clock Generator

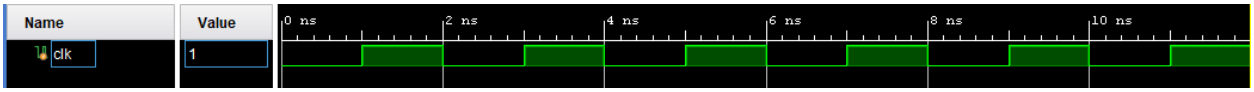
Module under test: Clock

Verifies that the clock generator produces a stable periodic signal with the correct period and 50% duty cycle.

Signals monitored

- clk - periodic clock output, period = 2 ns

Waveform



Result

The waveform shows stable and predictable clock behavior, ensuring reliable timing for all synchronous operations within the processor.

6.2 Counter (Program Counter)

Module under test: Counter

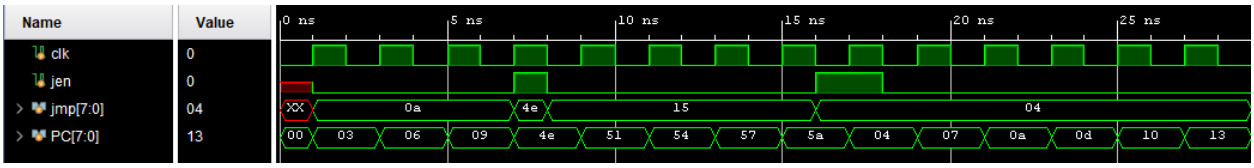
Verifies that the PC increments by 3 on each clock cycle during normal execution, and correctly loads a new address when jmp_en is asserted. Rollover behavior at PC=255 was also observed.

Signals monitored

- PC[7:0] - current program counter value
- jmp_en - jump enable input

- jmp[7:0] - jump target address

Waveform



Result

The waveform confirms correct jump behavior aligned with clock timing, ensuring that control flow changes occur at the appropriate cycle without timing errors.

6.3 Register File

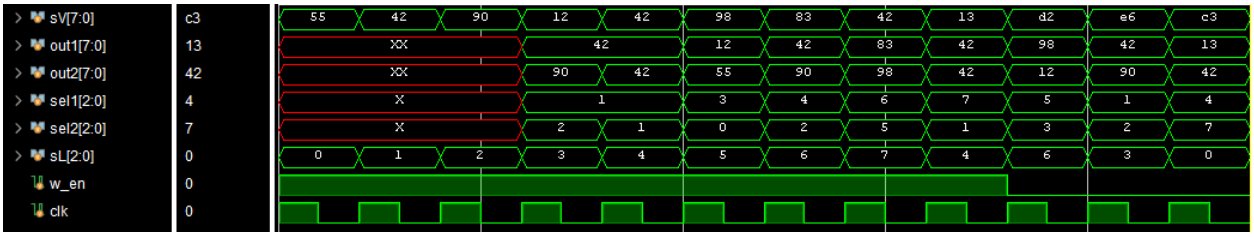
Module under test: Reg_file

Verifies asynchronous read behavior (outputs update immediately on select line change) and synchronous write behavior (data captured on clock edge with w_en asserted). Checks that writes to one register do not affect others.

Signals monitored

- out1[7:0] - read port 1 output
- out2[7:0] - read port 2 output
- w_en - write enable
- save[7:0] - write data
- sav_loc[2:0] - write address
- sel1[2:0], sel2[2:0] - read select lines

Waveform



Observations

The first few sections of out1 and out2 along with sel1 and sel2 show high impedance as the values haven't been decided by then, but from the fourth value onwards, the correct outputs are shown for the selected lines

Result

The select lines correctly route the chosen register values to the output, confirming accurate operand selection during execution.

6.4 ALU

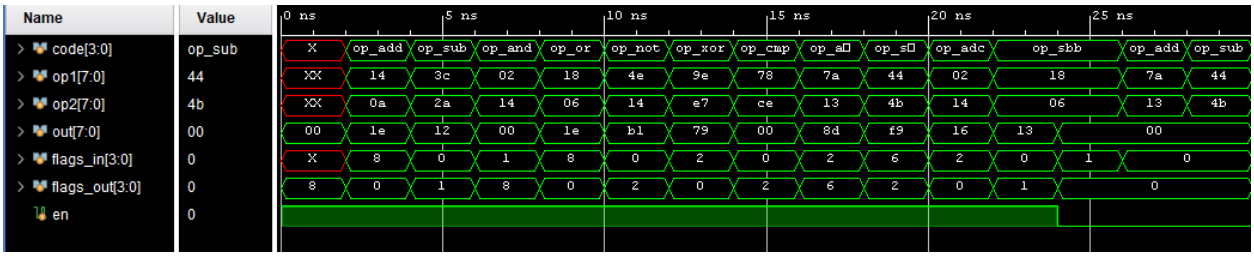
Module under test: ALU

Verifies all 11 operations including unsigned add/sub, logical operations, signed add/sub with overflow detection, and multi-word arithmetic with carry/borrow. Flag outputs are verified for each operation.

Signals monitored

- out[7:0] - result
- flagsout[3:0] - Z, V, N, C flags
- Op_code[3:0] - operation select
- op1[7:0], op2[7:0] - operands
- en - enable

Waveform



Result

The waveform demonstrates correct operation across all ALU functions, with no output generated when the enable signal is inactive. All computations are deterministic and match expected results.

6.5 Conditional Unit

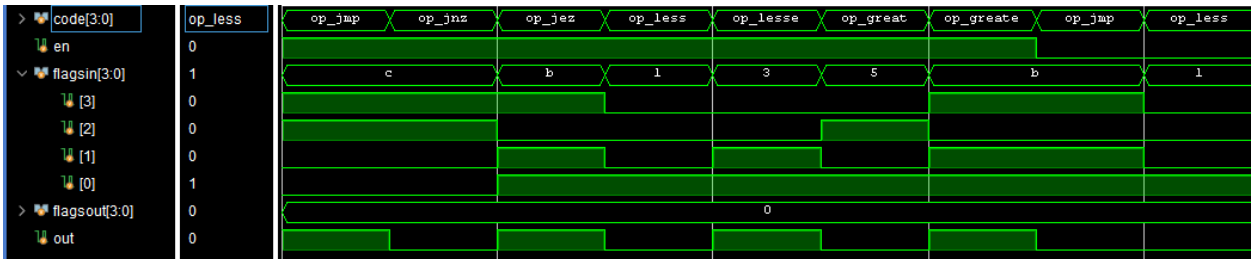
Module under test: Cond

Verifies all 7 branch conditions against known flag states. Confirms that jmp is asserted only when the condition is satisfied, and that flagsout is cleared to zero after every conditional evaluation.

Signals monitored

- jmp - branch taken output
- flagsin[3:0] - input flags
- flagsout[3:0] - flags after evaluation
- Op_code[3:0] - condition select
- en - enable

Waveform



Result

The output is correctly enabled based on condition evaluation, with appropriate flag usage. This validates proper implementation of conditional branching logic.

6.6 Stack

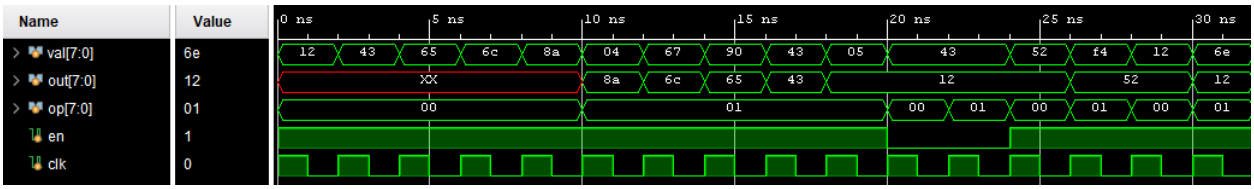
Module under test: Stack

Verifies push and pop sequencing: confirms the stack pointer increments on push and decrements on pop, that popped values match the LIFO order of previously pushed values, and that the stack pointer does not underflow below 0 or overflow above 255.

Signals monitored

- out[7:0] - popped value
- sp[7:0] - stack pointer (internal)
- val[7:0] - push input
- pp - push/pop select
- en - enable

Waveform



Result

The waveform verifies correct LIFO behavior of the stack, demonstrating reliable push and pop operations.

6.7 Full System - Top-Level Integration

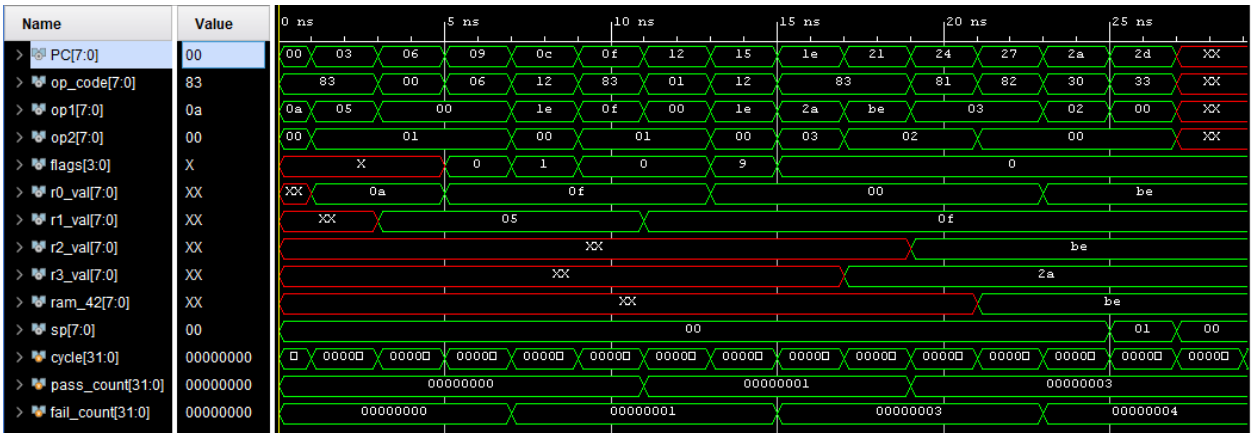
Module under test: Top (full system)

End-to-end verification of the integrated processor executing a 17-instruction test programme. The programme exercises immediate load, ALU add, CMP+conditional branch (with branch taken, skipping 5 NOP instructions), memory store and load, and stack push/pop-to-register. All results are checked automatically by the testbench.

Signals monitored

- PC[7:0] - program counter
- op_code[7:0], op1[7:0], op2[7:0] - instruction fetch
- r0-r3 (register file internal) - register values
- flags[3:0] - flag register
- RAM[5] - memory cell used in store/load test
- sp[7:0] - stack pointer
- pass_count, fail_count - testbench self-check counters

Waveform



Result

The processor executes instructions from the programme.mem file correctly, with all modules interacting as expected. The overall system behavior matches the intended design, with no timing inconsistencies or unintended behaviors were observed during simulation.

7. Known Limitations

The following limitations are acknowledged in the current revision of the Astra architecture. Several are intentional tradeoffs made in the interest of design simplicity; others are noted as areas for improvement in future revisions.

7.1 Program Memory Capacity

The programme memory is 256 bytes and instructions are 3 bytes each, yielding a maximum program size of approximately 85 instructions. This is sufficient for demonstration programs but limits the complexity of programs that can be run without modification to the architecture.

7.2 Memory Addressing

All RAM accesses are register-indirect: the address must be pre-loaded into a register before a load or store can be executed. There is no immediate memory addressing mode, which costs at least one additional instruction per memory access compared to architectures that support direct addressing.

7.3 Move Mode Encoding Efficiency

In move mode (op_code[7]=1), only 3 of the 8 op_code bits are used (bit[7] and bits[1:0]). The remaining 5 bits are unused. This is a known inefficiency - the fixed instruction format was prioritized over encoding density to keep the decoder simple. This will be revisited in the 16-bit revision.

7.4 No Hardware Call/Return

There is no dedicated CALL instruction. Subroutine calls must be implemented in software by manually pushing the return address onto the stack before jumping. The POPC instruction (pop to program counter) provides the return mechanism. This adds 2 instructions per call site but avoids the complexity of a hardwired call mechanism.

7.5 Simulation-Only Clock

The Clock module uses a simulation-only construct (forever #1 clk = ~clk) that cannot be synthesized. A real implementation would require an external clock input or an on-chip PLL. This is an intentional constraint since the architecture is not intended for physical synthesis in its current form.

7.6 No Reset Signal

There is no explicit reset port. The program counter and registers initialize to zero through simulation initial values rather than a hardware reset line. A synthesizable version would require a synchronous or asynchronous reset.

8. Conclusion

Astra demonstrates that a functional, self-consistent processor architecture can be constructed from a small set of well-defined modules operating on a single-cycle execution model. The design successfully executes programs covering arithmetic, conditional branching, memory access, and stack operations within an 8-bit address space.

The architecture makes several deliberate tradeoffs - fixed instruction width, register-indirect addressing, software-managed subroutine calls, and a simulation-only clock - each of which simplifies a different aspect of the hardware at the cost of some programming efficiency. These tradeoffs are appropriate for an experimental study platform and are well-understood as areas for improvement in future revisions.

Simulation results confirm correct execution of all tested instruction paths. The modular SystemVerilog implementation provides a clean basis for the planned 16-bit extension, in which many of the current limitations will be addressed.

Appendix A: Instruction Encoding Quick Reference

The following table provides a compact encoding reference for all Astra instructions. All values are given in binary and hexadecimal. Each instruction occupies 3 bytes in program memory.

op_code Bit Layout

bit[7]	bit[6]	bit[5]	bit[4]	bit[3]	bit[2]	bit[1]	bit[0]
mode	*(free)*	*unit[1]*	*unit[0]*	*op[3]*	*op[2]*	*op[1]*	*op[0]*

Register Index Encoding

Signal	Description
`r0`	000 (0x00)
`r1`	001 (0x01)
`r2`	010 (0x02)
`r3`	011 (0x03)
`r4`	100 (0x04)
`r5`	101 (0x05)
`r6`	110 (0x06)
`r7`	111 (0x07)

Flag Register Layout

Signal	Description
`flags[3]`	Z - Zero flag. Set when the result of an operation is zero.
`flags[2]`	V - Overflow flag. Set on signed arithmetic overflow.
`flags[1]`	N - Negative flag. Set when the result MSB is 1 (signed negative).
`flags[0]`	C - Carry flag. Set on unsigned carry out or borrow.

Module Wise Code

Clock

```
module Clock(clk);
 output logic clk;
initial begin
   clk = 0;
   forever #1 clk = ~clk;
   end
endmodule
```

Counter

```
module Counter(jmp_en,jmp,PC,clk);
   input jmp_en,clk;
   input [7:0] jmp;
   logic [7:0] count = 8'b0;
   output [7:0] PC;

   always_ff @ (posedge clk) begin
      if(jmp_en)
         begin
         count <= jmp;
         end
      else begin
         count <= count + 8'd3; //default instruction size is 3 bytes, hence counter increases by 3
to run properly
         end
   end
   assign PC = count;
endmodule
```

Programme

```
module Programme(count,op_code,op1,op2);
   input   logic   [7:0]   count;
   output  logic   [7:0]   op_code,op1,op2;
         logic   [7:0]   prog [255:0];
   initial begin
   $readmemb("programme.mem", prog);
   end

   assign  op_code =   prog[count];
   assign  op1     =   prog[count+1];
   assign  op2     =   prog[count+2];
endmodule
```

Register File

```
module Reg_file(w_en,save,sav_loc,sel1,sel2,out1,out2,clk);
   input   logic          w_en,clk;            // en = enable; any -> enable save/output = save => 1,out
=> 0; clock -> clk = clock
   input   logic   [7:0]   save;              // any -> save = save value;
   input   logic   [2:0]   sel1, sel2, sav_loc;// Program -> sel1,sel2,sav_loc = select lines 1,2 and
save location
   output  logic   [7:0]   out1, out2;        // any <- out1,out2 = output data from regs[sel1,sel2] to
wherever
          logic   [7:0]   regs [7:0];        // reg list. 8 regs of width 8

       always_comb begin   //instant reads
                out1 = regs[sel1]; //outputs to bus1
                out2 = regs[sel2]; //outputs to bus2
             end
       always_ff @ (posedge clk)begin  //clocked writes
             if(w_en) begin
                regs[sav_loc] <= save;
             end
             else
                regs[sav_loc] <= regs[sav_loc];
          end
endmodule
```

ALU

```
import def_cpu::*;

module ALU(Op_code,op1,op2,out,en,flagsin,flagsout);
   input math_op Op_code;
   input [7:0] op1,op2;
   input logic [3:0] flagsin;
   input en;
   output logic [7:0] out;
   output logic [3:0] flagsout;
   logic [8:0] temp;
   always_comb begin
      flagsout = 4'b0000;
      out = 8'd0;
      if(en) begin
                case(Op_code)
                    op_add: begin
                          temp = op1 + op2;
                          out = temp[7:0];
                          flagsout[0] = temp[8];
                          flagsout[1] = 0;
                          flagsout[2] = 0;
                          flagsout[3] = (out == 8'd0);
                          end
                    op_sub: begin
```

```
        temp = op1 - op2;
        out = temp[7:0];
        flagsout[0] = ~temp[8];
        flagsout[1] = 0;
        flagsout[2] = 0;
        flagsout[3] = (out == 8'd0);
        end
 op_and: begin
        out = op1 & op2;
        flagsout[0] = 0;                    //carry
        flagsout[1] = out[7];                //negative
        flagsout[2] = 0;                    //overflow
        flagsout[3] = (out == 8'd0);
        end
 op_or : begin
        out = op1 | op2;
        flagsout[0] = 0;                    //carry
        flagsout[1] = out[7];                //negative
        flagsout[2] = 0;                    //overflow
        flagsout[3] = (out == 8'd0);
        end
 op_not: begin
        out = ~op1;
        flagsout[0] = 0;                    //carry
        flagsout[1] = out[7];                //negative
        flagsout[2] = 0;                    //overflow
        flagsout[3] = (out == 8'd0);
        end
 op_xor: begin
        out = op1 ^ op2;
        flagsout[0] = 0;                    //carry
        flagsout[1] = out[7];                //negative
        flagsout[2] = 0;                    //overflow
        flagsout[3] = (out == 8'd0);
        end
 op_cmp: begin
        temp = op1-op2;
        flagsout[0] = ~temp[8];                //carry
        flagsout[1] = temp[7];                //negative
        flagsout[2] = (op1[7] == op2[7] && op1[7] != temp[7]);   //overflow
        flagsout[3] = (temp == 8'd0);
        end
op_adds: begin
        temp = op1 + op2;
        out = temp[7:0];
        flagsout[0] = temp[8];                //carry
        flagsout[1] = out[7];                //negative
        flagsout[2] = (op1[7] == op2[7] && op1[7] != out[7]);   //overflow
        flagsout[3] = (out == 8'd0);
        end
op_subs: begin
```

```
                    temp = op1 - op2;
                    out = temp[7:0];
                    flagsout[0] = ~temp[8];                    //carry
                    flagsout[1] = out[7];                      //negative
                    flagsout[2] = (op1[7] != op2[7] && op1[7] != out[7]);   //overflow
                    flagsout[3] = (out == 8'd0);
                    end
                op_adc: begin
                    temp = op1 + op2 + flagsin[0];
                    out = temp[7:0];
                    flagsout[0] = temp[8];
                    flagsout[1] = 0;
                    flagsout[2] = 0;
                    flagsout[3] = (out == 8'd0);
                    end
                op_sbb: begin
                    temp = op1 - op2 - ~flagsin[0];
                    out = temp[7:0];
                    flagsout[0] = ~temp[8];
                    flagsout[1] = 0;
                    flagsout[2] = 0;
                    flagsout[3] = (out == 8'd0);
                    end
               default: begin
                    out = 8'b0;
                    flagsout[0] = 0;                           //carry
                    flagsout[1] = out[7];                      //negative
                    flagsout[2] = 0;                           //overflow
                    flagsout[3] = (out == 8'd0);
                    end
            endcase
        end
    end
endmodule
```

Condition Block

```
import def_cpu::*;

module Cond(Op_code,flagsin,flagsout,en,jmp);
   input cmp_op Op_code;
   input logic en;
   input  logic [3:0] flagsin;
   output logic [3:0] flagsout;
   output logic jmp;
   logic g1;
   xor (g1,flagsin[1],flagsin[2]);
   always_comb begin
   jmp=0;
      if(en) begin
```

```
        case(Op_code)
            op_jmp   : jmp=1;
            op_jnz   : if(~flagsin[3])begin jmp=1;end
            op_jez   : if(flagsin[3])begin jmp=1;end
            op_less  : if(flagsin[1] ^ flagsin[2])begin jmp=1;end
            op_lesse : if((flagsin[1] ^ flagsin[2])|| flagsin[3])begin jmp=1;end
            op_great : if(!flagsin[3] && !g1)begin jmp=1;end
            op_greate: if(flagsin[3] || g1)begin jmp=1;end
        endcase
    end
    flagsout = 4'b0;
  end
endmodule
```

Xtra Block

```
import def_cpu::*;
module Xtra(op1,op,op2,out,en,flagsin,flagsout);
  input                en;
  input   xtr_op       op;
  input   logic   [3:0]   flagsin;
  input   logic   [7:0]   op1,op2;
  output  logic   [7:0]   out;
  output  logic   [3:0]   flagsout;
  logic           [2:0]   sh;
  logic           [8:0]   ext;
  always_comb begin
    if(en) begin
        sh = op2[2:0];
        case(op)
            op_shr: begin
                out = op1 >> sh;
                end
            op_shl: begin
                out = op1 << sh;
                end
            op_ror: begin
                out = (op1 << (8-sh)) | (op1 >> sh);
                end
            op_rol: begin
                out = (op1 >> (8-sh)) | (op1 << sh);
                end
           default: out = op1;
        endcase
    end
  end
endmodule
```

Stack

```
module Stack(val,clk,en,out,op);
   input [7:0] val,op; // any -> val = input value; counter -> cntr = counter value for call/ret
functions
   input en,clk; //cond -> en = enable; clock -> clk = clock; pp = 0 => push, 1 => pop;
   logic [7:0] sp = 8'd0; //stack pointer
   logic [7:0] regs [255:0]; //reg_file 255 regs with 8 bit storage
   output logic [7:0] out; //jump loc -> program Counter
  always_ff @ (posedge clk)
   begin
      if(en) begin
            case(op[1:0])
            2'b00:   begin //push
                  if(sp < 8'd255)
                     begin
                        regs[sp] <= val; //stores input
                        sp <= sp + 1; //update stack pointer
                     end
                    end
            2'b01:begin //pop
                  if(sp > 8'd0)
                     begin
                        out <= regs[sp-1];
                        sp <= sp - 1;
                     end
               end
            2'b11:begin //pop counter
                  if(sp > 8'd0)
                     begin
                        out <= regs[sp-1];
                        sp <= sp - 1;
                     end
               end
         endcase
      end
   end
endmodule
```

Ram

```
module Ram(sv,s,clk,out,read,en);
   input   logic   [7:0]   sv, s; // sv = save value, s = save loc
   input   logic           clk,read,en; //read = 1 -> out; read = 0 -> save
   output  logic   [7:0]   out;
         logic   [7:0]   sram    [255:0];

   always_ff @ (posedge clk) begin
      if(en && !read)
            sram[s] <= sv;
```

```
        end
    always_comb begin
        if(en) begin
            out = 8'd0;
            if(read)
                out = sram[s];
        end
        else
            out = 8'd0;
    end
endmodule
```

TOP Module

```
import def_cpu::*;

module Top();
math_op alu_op;
cmp_op comp_op;
xtr_op xtra_op;
logic clk,jmp_cond,en1,en2,en3,en4,jmp,w_en,call,pp,s_en,c_en,a_en,x_en,ram_en,ram_read;
logic [7:0] op_code,op1,op2,jmp_loc,res_alu,res_salu,res_sft,SP,stack_out,ramsave,ramloc;
logic [7:0] count,out1,out2,out_ram,sav_val,sav_loc;
logic [3:0] flagsin,flagsalu,flagscond,flagsxtra,flagsout;     //flags = 0000 -> z,o,n,c
Clock      m0(.clk(clk));                                                                    //clk
Counter     m1(.jmp_en(jmp),.jmp(jmp_loc),.PC(count),.clk(clk));
//jmp_en,jmp,PC,clk
Programme   m2(.count(count),.op_code(op_code),.op1(op1),.op2(op2));
//count,op_code,op1,op2
Reg_file
m3(.w_en(w_en),.save(sav_val),.sav_loc(sav_loc[2:0]),.sel1(op1[2:0]),.sel2(op2[2:0]),.out1(out1
),.out2(out2),.clk(clk));    //w_en,save,sav_loc,sel1,sel2,out1,out2,clk
ALU
m4(.en(a_en),.Op_code(alu_op),.op1(out1),.op2(out2),.out(res_alu),.flagsin(flagsin),.flagsout(fla
gsalu));                  //(Op_code,op1,op2,out,en,flagsin,flagsout)
Cond
m6(.en(c_en),.Op_code(comp_op),.flagsin(flagsin),.flagsout(flagscond),.jmp(jmp_cond));
//Op_code,flagsin,flagsout,en,jmp
Xtra
m7(.op(xtra_op),.op1(out1),.op2(out2),.out(res_sft),.flagsin(flagsin),.flagsout(flagsxtra),.en(x_en)
);                    //op1,op,op2,out,en,flagsin,flagsout
Stack      m8(.val(out1),.clk(clk),.en(s_en),.op(op_code),.out(stack_out));
//op_code,val,cntr,clk,en,out
Ram        m9(.sv(ramsave),.s(ramloc),.clk(clk),.out(out_ram),.read(ram_read),.en(ram_en));
//sv,s,clk,out,s_o,en

always_ff @ (posedge clk) begin
    flagsin <= flagsout; //flags set for next cycle
    end
```

```
always_comb begin
   alu_op = math_op'(4'd0); comp_op = cmp_op'(4'd0); xtra_op = xtr_op'(3'd0);
   a_en = 0;c_en = 0;s_en = 0;x_en = 0; //enable pins initialization
   ram_en = 0; ramsave = 8'd0;ramloc=8'd0; ram_read=0; //mov initialization
   sav_val=8'd0;w_en=0;sav_loc = 'x; //regs initialization
   jmp=0;jmp_loc=8'd0; //stack initialization
   flagsout = flagsin; //flags initialization for cycle
   if (!op_code[7]) // calc mode
      begin
         case(op_code[5:4])
         2'b00:  begin
                  alu_op = math_op'(op_code[3:0]);
                  a_en = 1'b1;
                  sav_val = res_alu;
                  sav_loc = op1;
                  flagsout = flagsalu;
                  if(op_code[3:0] == 4'b0110)
                     w_en=0;
                  else
                     w_en=1;
               end
         2'b01:  begin
                  w_en = 0;
                  comp_op = cmp_op'(op_code[3:0]);
                  c_en = 1'b1;
                  jmp = jmp_cond;
                  jmp_loc = op1;
                  flagsout = flagscond;
               end
         2'b10:  begin
                  xtra_op = xtr_op'(op_code[2:0]);
                  x_en = 1'b1;
                  sav_val = res_sft;
                  sav_loc = op1;
                  w_en = 1;
                  flagsout = flagsxtra;
               end
         2'b11:  begin
               w_en = 0;
               s_en = 1'b1;
               if(op_code[1:0] == 2'b11)   begin
                     jmp = 1;
                     jmp_loc = stack_out;
                     end
               else if(op_code[1:0] == 2'b01)    begin
                     w_en = 1;
                     sav_val = stack_out;
                     sav_loc = op1;
                     end
               end
      endcase
```

```
        end
    else begin //mov mode
            case(op_code[1:0])
            2'b00:   begin
                        w_en = 1;
                        sav_loc = op2;
                        sav_val = out1;
                     end
            2'b01:   begin
                        w_en = 0;
                        ram_en = 1;
                        ramloc = out1;
                        ram_read=0;
                        ramsave = out2;
                     end
            2'b10:   begin
                        ram_en = 1;
                        ramloc = out1;
                        ram_read = 1;
                        w_en = 1;
                        sav_loc = op2;
                        sav_val = out_ram;
                     end
            2'b11:   begin
                        w_en = 1;
                        sav_val = op1;
                        sav_loc = op2;
                     end
        endcase
    end
end

endmodule
```

Def_cpu

```
package def_cpu;

    typedef enum logic [3:0]{
    op_add,          // a + b
    op_sub,          // a - b
    op_and,          // a & b
    op_or,           // a | b
    op_not,          // ~a
    op_xor,          // a ^ b
    op_cmp,          // a _ b (updates flags)
    op_adds = 4'b1000,  // a + b
    op_subs = 4'b1001,  // a - b
    op_adc  = 4'b1010,  // add with carry
    op_sbb  = 4'b1011   // sub with borrow
```

```
    }math_op;

    typedef enum logic [3:0]{ //always in the form a __ b
    op_jmp,         //jump regardless
    op_jnz,         //jump if a not zero
    op_jez,         //jump if a is zero
    op_less,        //jump if a <  b
    op_lesse,       //jump if a <= b
    op_great,       //jump if a >  b
    op_greate       //jump if a >= b
    }cmp_op;

    typedef enum logic [2:0]{
    op_shr, //shift right
    op_shl, //shift left
    op_ror, //roll right
    op_rol //roll left
    }xtr_op;

    localparam int REG_COUNT = 8;


endpackage
```